

PM Controller

Phase 2 — Completion record · the task board, columns, drag, role-gated transitions

Owner Apex Plumbing and Mechanical Services SC **Merged** PR #2 · 22 September 2026 **Status** Milestone met

Tests 69 of 69

1. The milestone

"A Lead cannot move a task to Accomplished."

Asserted in the database, where it is enforced, and made true through the app. The board hides the move; the policy refuses it; and if the two ever disagree, the database wins and the person sees an error rather than a silent failure.

NEW ASSERTION	WHAT IT PROTECTS
A Lead moves his card between two columns of the same core status	The status does not change — a custom column is cosmetic
A column cannot be set without the status that matches it	The pair must travel together, so reporting never reads a column name
A Lead cannot see another project's columns at all	Row security
A Manager can see both projects, but cannot drop a card across them	The trigger, as an independent second layer
A Lead cannot move a card into Accomplished	The Phase 2 milestone
A Super can make that same move, and it writes its own audit row	Sign-off authority, and the audit trail

2. What was built

PIECE	DECISION, AND WHY
Columns grouped by core status	Ten flat columns on a tablet is a scroll with no landmarks. Grouping makes the locked structure visible, so a Super can see that "Waiting on GC" is still In Progress without being told.
The transition engine	A move names a column ; the status is derived from it, never the reverse. Both travel in one update, which is what the database trigger requires anyway — so the two layers cannot drift apart.
Optimistic moves	A board that pauses on every drag feels broken on site Wi-Fi. The card lands immediately; a refusal restores it and says why.
Tap-to-move sheet	The phone's only path, and the better one with gloves. It does what dragging cannot: lists the moves that are <i>closed</i> to you and says why, so a Lead learns the rule once instead of discovering it as a failed drag.
Ticket detail with history	The audit trail is the product. Read-only here and read-only in the database — <code>task_events</code> has no client write policy at all.
<code>Ticket</code> , not <code>Task</code>	Swift Concurrency owns that name. SPEC already calls them tickets, so the domain language and the compiler agree.

3. Decision log

Merge Phase 1 and start Phase 2.

PR #1 merged with a merge commit rather than a squash — three of its six commits record CI failures and their causes, and on a project whose own principle is that the audit trail is the product, discarding that would be the wrong instinct. Phase 1 completion record and Phase 2 action plan issued before any board code was written.

apexmech.com is ours.

Bundle identifier `com.apexmech.pmcontroller` confirmed correct. The reverse-DNS name now matches a domain the company actually controls, which is the one thing that is awkward to change after App Store submission.

— built the board —

Five views, two model files and a store, written without a local Swift toolchain. It compiled on the first CI attempt, which was not expected. The actor-isolation fixes made by reading the diff before pushing rather than waiting for the compiler — `BoardView.init` marked `@MainActor` , and the store lookup moved inside an isolated `Task` in the drop handler — are what strict concurrency would otherwise have rejected.

Two test bugs, caught before they shipped

Both made assertions pass while testing nothing, which is worse than having no test at all.

The first cut reused a Lead that the suite deactivates earlier. An inactive user matches no policy, so every statement touched zero rows — and a statement that changes nothing reads as a *pass* for anything asserting a refusal. Four assertions were green for the wrong reason. Fixed with an active Lead and a ticket of the section's own.

The second conflated two separate cross-project protections. A Lead cannot name another project's column because row security hides it, so a subquery yields null — and a null column simply means "not placed yet". The trigger is an independent layer. It is now asserted as a Manager, who can see both projects, so the layer that would matter if a policy were ever loosened is actually covered.

4. Verified, and not

VERIFIED	NOT VERIFIED
• 69/69 assertions against a real Postgres 16 • The board's rules hold where they are enforced • The Swift compiles for iPad at the iOS 18 floor • The app launches and renders the board as each role • Screenshots as Super and as Lead, on every push	• No migration has run against a live Supabase project • Nobody has signed in with a real credential • The Edge Function has never been deployed • Behaviour on a physical iPad • Whether dragging a card works with gloves on

The same caution as Phase 1, now one phase larger. Every guarantee in this record holds against a test database. Two phases of work are now stacked on something nobody has signed into. Creating the Supabase project is a fifteen-minute job, and it converts a great deal of "should work" into "does work" — including the one thing CI structurally cannot check, which is whether the policies behave the same way through PostgREST as they do through psql.

5. Carried into Phase 3

- ☒ Board renders the seeded project, grouped by core status
- ☒ A move derives status from the target column, never the reverse
- ☒ A Lead cannot move a card into Accomplished — the milestone
- ☒ A refused move snaps back and explains itself
- ☒ Create a ticket; open one; read its history
- ☒ Screenshots cover the board as Super and as Lead
- ☒ CI green on all three jobs
- ☐ **Create the Supabase project** and apply `0001` , `0002` , then seed
- ☐ **Deploy `admin-create-user`** and create the first real Manager
- ☐ **Sign in on a real iPad** and move one card
- ☐ Show the board to a Lead before Phase 3 builds time capture on top of it
- ☐ Data-ownership terms with PCM in writing
- ☐ Decide whether area and system stay free text or become a fixed list per project

6. Phase 3 — what comes next

SPEC §12: *time capture* — *crew selection, per-person rows*. Done when **one session with three crew writes three rows**.

THE RULE IT HAS TO HONOUR	WHAT THAT MEANS IN THE BUILD
Hours are per person, never typed at task level (principle 3)	The Lead sets a start and an end and picks who was there. The app writes one <code>time_entries</code> row per person. Task hours are always a SUM — there is no field anywhere that accepts a number of hours.
The assigned crew is not the crew that turned up	<code>task_crew</code> and <code>time_entries</code> are already separate tables for exactly this reason. An assigned man who was not there must not accrue hours.
Rework hours flag themselves	A session opened from Rework sets <code>is_rework</code> automatically. Nobody has to remember to tag anything.
One open session per person	Already enforced by a unique index. The app has to handle that refusal gracefully — a man cannot be clocked onto two jobs at once.

7. Ideas worth keeping

IDEA	WHY IT CAME UP
The tap-to-move sheet as a teaching tool	Built for gloves and phones, but it turned out to be the clearest explanation of the role matrix in the whole app — it names every move you cannot make and why. Worth keeping visible rather than hiding it once dragging works.

Screenshots as a review artifact	Comparing the same board as two roles, side by side, is a faster review than reading a policy file. It would also demonstrate the access rules to a pilot customer in about ten seconds.
Tests that can pass for the wrong reason	Both Phase 2 test bugs had the same shape: a statement that touches zero rows looks like a refusal. Worth a standing habit — every "must be refused" assertion should also confirm the row is unchanged.